# Реферат

Реляционные СУБД предоставляют комплексный инструментарий для создания и работы с базами данных. Они позволяют не только проектировать структуру данных и выполнять операции с ними через запросы, но и обеспечивают надежную защиту хранимой информации. В данной работе расширяется база данных с помощью таких элементов СУБД, как представления, права пользователей, триггеры, транзакции, функции и процедуры. работа выполнялась с помощью СУБД MySQL 8.0 для взаимодействия с указанными элементами на языке SQL. Реализованные в данной работе средства могут быть использованы в работе с исходной базой данных для мониторинга домашних и бездомных собак.

# Содержание

**Реферат** **2**

**Введение** **4**

**1 Постановка задачи** **5**

**2 Представления** **6**
2.1 Постановка задачи . . . . . 6
2.2 Реализация . . . . . 6

**3 Триггеры** **10**
3.1 Постановка задачи . . . . . 10
3.2 Реализация . . . . . 10

**4 Права пользователей** **17**
4.1 Постановка задачи . . . . . 17
4.2 Реализация . . . . . 17

**5 Функции и процедуры** **22**
5.1 Постановка задачи . . . . . 22
5.2 Реализация . . . . . 22

**6 Транзакции** **27**
6.1 Постановка задачи . . . . . 27
6.2 Реализация . . . . . 27

**Заключение** **30**

**Список использованных источников** **31**

**Приложение А. Исходный код функции** **32**

**Приложение Б. Исходный код процедуры** **33**

# Введение

Базы данных играют важную роль в организации хранения, обработки и анализа информации в разнообразных сферах деятельности. Они являются основным инструментом для систематизации и надежного хранения больших объемов информации. Базы данных обеспечивают структурированный доступ к данным, позволяют эффективно выполнять поиск, обновление и анализ информации.

Современные СУБД предоставляют для этого развитый инструментарий. Их ключевые возможности включают не только определение структуры данных и манипулирование ими через запросы, но и комплексные механизмы для обеспечения безопасности и целостности. Именно такие средства, как представления, транзакции, процедуры и триггеры, позволяют перейти от простого хранения информации к построению надежных и безопасных информационных систем.

Целью данной работы является расширение ранее разработанной базы данных с помощью ключевых элементов управления данными: представлений, пользователей, триггеров, транзакций, функций и процедур. Практическая реализация и анализ их взаимодействия выполняются на примере СУБД MySQL 8.0, а полученные результаты направлены на совершенствование функциональности и надежности базы данных для мониторинга собак.

# 1 Постановка задачи

Цель данной работы : расширить ранее разработанную базу данных с помощью следующих элементов СУБД: представления, пользователи, триггеры, транзакции, функции и процедуры.

Задачи.

1. Реализовать представление и выполнить с ним ряд различных запросов.
2. Реализовать несколько тригеров на отображение изменений информации в новой таблице при влиянии на исходные таблицы.
3. Реализовать пользователей с различными правами и проверить их ограничения в работе с таблицами.
4. Реализовать функцию и процедуру для работы с данными имходной базы данных.
5. Выбрать одну анамалию и уровень изоляции транзакций и изучить поведение данной аномалии в рамках выбранного уровня изоляции.

# 2 Представления

## 2.1 Постановка задачи

В данной работе необходимо реализовать ряд следующих задач.

1. На основании схемы базы данных прошлого семестр создать представление для запроса, содержащего в себе агрегатные функции, SELECT, JOIN и GROUP BY.

2. Написать простой запрос, использующий созданное представления.

3. Написать сложный запрос для данного представления, использующий JOIN и GROUP BY.

4. Продемонстрировать доступность представления только для чтения.

## 2.2 Реализация

Исходя из схемы базы данных прошлого семестра для создания представления в данной работе был выбран следующий запрос: для каждого производителя посчитать количесвто чиппированных собак и число ветеринарных клиник, в которых собаки лечились.

Выбор был сделан в пользу данного запроса, так как его реализация содержит агрегатные функции COUNT, JOIN и GROUP BY, что соответствует условию данной работы.

Код создания представления view_1 показан в листинге 1.

Листинг 1. Создание представления view_1

```
CREATE VIEW view_1 as
SELECT
manufacturer.id_manufacturer,
manufacturer.name_manufacturer AS name_manufacturer,
COUNT(DISTINCT m.id_dog) AS count_dog ,
COUNT(DISTINCT v.id_vet_clinic) AS count_vet_clinic
FROM
manufacturer
JOIN (
SELECT id_microchip, id_manufacturer, id_dog
FROM microchip
WHERE id_dog IS NOT NULL
)  m ON manufacturer.id_manufacturer = m.id_manufacturer
JOIN (
SELECT id_dog, id_veterinarian
FROM medical_service_record
) ms ON m.id_dog = ms.id_dog
JOIN (
SELECT id_veterinarian, id_vet_clinic
FROM veterinarian
) v ON ms.id_veterinarian = v.id_veterinarian
GROUP BY  manufacturer.id_manufacturer, manufacturer.name_manufacturer;
```

Пример вывода данных созданного представления для первых 18 производителей показан на рисунке 1.

| id_manufacturer | name_manufacturer | count_dog | count_vet_clinic |
|---|---|---|---|
| 1 | Производитель1 | 496 | 613 |
| 2 | Производитель2 | 475 | 616 |
| 3 | Производитель3 | 489 | 609 |
| 4 | Производитель4 | 425 | 611 |
| 5 | Производитель5 | 449 | 615 |
| 6 | Производитель6 | 437 | 611 |
| 7 | Производитель7 | 471 | 618 |
| 8 | Производитель8 | 433 | 609 |
| 9 | Производитель9 | 430 | 615 |
| 10 | Производитель10 | 444 | 613 |
| 11 | Производитель11 | 428 | 608 |
| 12 | Производитель12 | 464 | 610 |
| 13 | Производитель13 | 458 | 609 |
| 14 | Производитель14 | 461 | 616 |
| 15 | Производитель15 | 435 | 619 |
| 16 | Производитель16 | 465 | 611 |
| 17 | Производитель17 | 440 | 611 |
| 18 | Производитель18 | 460 | 618 |

Рис. 1. Результат вывода представления

Также после создания представления можно проверить его наличие через информационную схему базы данных с помощью конструкции, представленной в листинге 2.

Листинг 2. Вывод всех VIEW базы данных

```
SELECT* FROM information_schema.views where TABLE_SCHEMA='dog_db';
```

После выполнения данной команды будут выведены все представления, имеющие отношение к базе данных, как показано на рисунке 2.

| TABLE_CATALOG | TABLE_SCHEMA | TABLE_NAME | VIEW_DEFINITION | CHECK_OPTION | IS_UPDATABLE | DEFINER | SECURITY_TYPE |
|---|---|---|---|---|---|---|---|
| def | dog_db | view_1 | select `dog_db`.`manufacturer`.`id_manufact... | NONE | NO | root@localhost | DEFINER |

Рис. 2. Вывод всех VIEW базы данных

В работе реализован следующий простой запрос, использующий представление view_1 во FROM. Данный запрос выполняет вывод данных представления согласно заданным условиям: микрочипами производителя чипированно не менее 450 собак и данным собакам оказана ммедицинская помощь не более чем в 615 клиниках.

Реализация данного запроса представлена в листинге 3.

Листинг 3. Простой запрос к представлению

```sql
SELECT
name_manufacturer,
count_dog,
count_vet_clinic
FROM view_1
WHERE count_dog <=450 and count_vet_clinic >615;
```

Результат работы реализованного запроса представлен на рисунке 3.

| name_manufacturer | count_dog | count_vet_clinic |
|---|---|---|
| Производитель15 | 435 | 619 |
| Производитель26 | 438 | 620 |
| Производитель34 | 446 | 616 |
| Производитель40 | 441 | 616 |
| Производитель43 | 439 | 616 |

Рис. 3. Результат работы простого запроса

Согласно поставленным задачам сложный запрос к представлению должен включать в себя агрегатные функции и JOIN. Исходя из данного условия для выоплнения задания был выбран следующий запрос: для каждого производителя посчитать количесвто чиппированных собак, число ветеринарных клиник, в которых собаки лечились и число услуг оказанных данным собакам в ветеринарных клиниках.

Представление уже содержит в себе информацию про собак и ветеринарные клиники, следовательно в запросе необходимо получить данные об оказанных медицинских услугах.

Реализация данного запроса представлена в листинге 4.

Листинг 4. Сложный запрос к представлению

```sql
SELECT
v.name_manufacturer,
v.count_dog,
v.count_vet_clinic,
COUNT(ms.id_record) AS count_medical_service_record
FROM
view_1 as v
JOIN microchip m ON v.id_manufacturer = m.id_manufacturer
JOIN medical_service_record ms ON m.id_dog = ms.id_dog
WHERE m.id_dog IS NOT NULL
GROUP BY v.name_manufacturer, v.count_dog, v.count_vet_clinic
ORDER BY count_medical_service_record DESC;
```

Результат работы данного запроса представлен на рисунке 4.

| name_manufacturer | count_dog | count_vet_clinic | count_medical_service_record |
|---|---|---|---|
| Производитель45 | 495 | 619 | 3503 |
| Производитель46 | 493 | 621 | 3486 |
| Производитель42 | 497 | 621 | 3485 |
| Производитель1 | 496 | 613 | 3435 |
| Производитель36 | 487 | 617 | 3427 |
| Производитель3 | 489 | 609 | 3410 |
| Производитель21 | 481 | 619 | 3368 |
| Производитель7 | 471 | 618 | 3337 |
| Производитель38 | 471 | 615 | 3314 |
| Производитель20 | 469 | 617 | 3308 |
| Производитель2 | 475 | 616 | 3293 |
| Производитель19 | 466 | 618 | 3292 |
| Производитель12 | 464 | 610 | 3291 |
| Производитель41 | 477 | 612 | 3290 |

Рис. 4. Результат сложного запроса к представлению

Основное свойство представлений это доступность только для чтения. Проверить данное свойство можно с помощью попытки применить операции на добавление, удаление и изменение данных в данном представлении. Код данных операций представлен в листинге 5.

Листинг 5. Проверка свойств представления

```
UPDATE view_1 set count_dog=20 where id_manufacturer =1;
DELETE FROM view_1 WHERE id_manufacturer = 1;
INSERT INTO view_1 (id_manufacturer, name_manufacturer, count_dog,
    count_vet_clinic)
VALUES (52, 'Производитель 52', 20, 30);
```

При выполнении каждой такой операции будет выведена ошибка, говорящая о недоступности данного представления для изменений. Вывод соответствующих ошибок представлен на рисунке 5.

| | | | | |
|---|---|---|---|---|
| 9 | 18:46:53 | UPDATE view_1 set count_dog=20 where id_manufacturer =1 | Error Code: 1288. The target table view_1 of the UPDATE is not updatable | 0.000 sec |
| 10 | 18:46:58 | DELETE FROM view_1 WHERE id_manufacturer = 1 | Error Code: 1288. The target table view_1 of the DELETE is not updatable | 0.000 sec |
| 11 | 18:47:02 | INSERT INTO view_1 (id_manufacturer, name_manufacturer, count_dog, count... | Error Code: 1471. The target table view_1 of the INSERT is not insertable-into | 0.000 sec |

Рис. 5. Ошибки при изменении представления

# 3 Триггеры

## 3.1 Постановка задачи

В данной работе необходимо реализовать ряд следующих задач.

1. На основании схемы базы данных прошлого семестра создать новую таблицу, хранящую некоторую совокупность данных из двух исходных таблиц.

2. Создать триггеры на отображение изменений информации в новой таблице при добавлении, изменении, удалении данных исходных таблиц.

3. Продемонстрировать результат работы триггеров.

## 3.2 Реализация

В данной работе необходимо создать новую таблицу breed_statistics содержащую данные о количестве собак каждой породы. Новая схема базы данных с добавленной таблицей представлена на рисунке 6.

Новая таблица основывается на двух исходных таблицах breed и dog. Следовательно необходимо реализовать триггеры на добавление, изменение и удаление данных в каждой из этих таблиц.

Код создания новой таблицы и код заполнения таблицы исходныи данными представлены в листинге 6.

Листинг 6. Создание новой таблицы

```
CREATE TABLE IF NOT EXISTS breed_statistics (
id_breed INT PRIMARY KEY REFERENCES breed(id_breed),
name_breed CHAR(44) NOT NULL,
dog_count INT NOT NULL DEFAULT 0
);

INSERT INTO breed_statistics (id_breed, name_breed, dog_count)
SELECT
b.id_breed,
b.name_breed,
COUNT(d.id_dog) as dog_count
FROM breed b
LEFT JOIN dog d ON b.id_breed = d.id_breed
GROUP BY b.id_breed, b.name_breed;
```

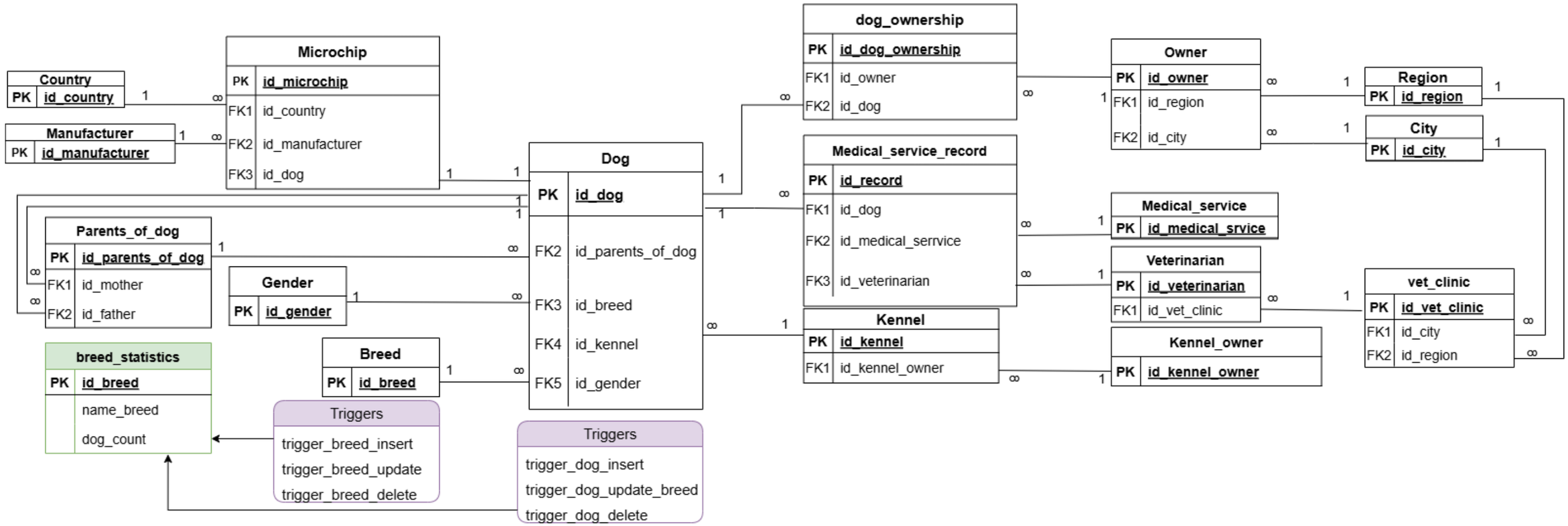


Рис. 6: Схема базы данных на английском языке языке

В листинге 7 представлено создание триггера trigger_breed_insert на добавление в словарь breed новой породы. В таком случае для каждой новой строки в таблице breed будет выполняться добавление данной породы в таблицу breed_statistics со счетчиком собак, равным 0. Команда DELIMITER позволяет писать многострочные триггеры, не прерывая их разделителем ";".

Листинг 7. Триггер на добавление породы

```
DELIMITER //

CREATE TRIGGER trigger_breed_insert
AFTER INSERT ON breed
FOR EACH ROW
INSERT INTO breed_statistics (id_breed, name_breed, dog_count)
VALUES (NEW.id_breed, NEW.name_breed, 0);

DELIMITIR ;
```

При изменении breed срабатывает триггер trigger_breed_update, представленный в листинге 8. Данный триггер в таблице breed_statistics обновляет названия соответствующих пород.

Листинг 8. Триггер на обновление таблицы breed

```
DELIMITER //
CREATE TRIGGER trigger_breed_update
AFTER UPDATE ON breed
FOR EACH ROW
UPDATE breed_statistics
SET name_breed = NEW.name_breed
WHERE id_breed = NEW.id_breed;

DELIMITIR ;
```

В листинге 9 показана реализация триггера trigger_breed_delete на удаление породы из таблицы breed. Данный триггер удаляет строку с необходимой породой согласно полю id_breed.

Листинг 9. Триггер на удаление породы

```
DELIMITER //

CREATE TRIGGER trigger_breed_delete
AFTER DELETE ON breed
FOR EACH ROW
DELETE FROM breed_statistics
WHERE id_breed = OLD.id_breed;

DELIMITER ;
```

При появлении новой записи в dog срабатывает триггер trigger_dog_insert. Данный триггер обновляет данные поля dog_count, соответствующей породы в таблице breed_statistics, увеличивая знаечние данного поля на 1. Реализация триггера trigger_dog_insert показана в листинге 10.

Листинг 10. Триггер на добавление собаки

```sql
DELIMITER //

CREATE TRIGGER trigger_dog_insert
AFTER INSERT ON dog
FOR EACH ROW
UPDATE breed_statistics
SET dog_count = dog_count + 1
WHERE id_breed = NEW.id_breed;

DELIMITER ;
```

В листинге 11 представлена реализация триггера на удаление собаки из таблицы dog. В таком случае уменьшается счетчик собак для соответствующей породы.

Листинг 11. Триггер на удаление собаки

```sql
DELIMITER //

CREATE TRIGGER trigger_dog_delete
AFTER DELETE ON dog
FOR EACH ROW
UPDATE breed_statistics
SET dog_count = dog_count - 1
WHERE id_breed = OLD.id_breed;

UPDATE breed_statistics
                SET dog_count = 0
WHERE id_breed = OLD.id_breed AND dog_count < 0;

DELIMITER ;
```

В листинге 12 представлен триггер trigger_dog_update_breed на изменение даных собаки в таблице dog. Данный триггер проверяет, изменена ли порода собаки, если да, то счетчик исходной породы уменьшается,а счетчик новой - увеличивается.

Листинг 12. Триггер на добавление собаки

```sql
CREATE TRIGGER trigger_dog_update_breed
AFTER UPDATE ON dog
FOR EACH ROW
IF OLD.id_breed != NEW.id_breed  THEN
UPDATE breed_statistics
SET dog_count = dog_count - 1
WHERE id_breed = OLD.id_breed;

UPDATE breed_statistics
SET dog_count = 0
WHERE id_breed = OLD.id_breed AND dog_count < 0;
END IF;

UPDATE breed_statistics
SET dog_count = dog_count + 1
WHERE id_breed = NEW.id_breed;

DELIMITER ;
```

Как и в случае представлений с помощью информационной схемы и команды, представленной в листинге 14 можно увидеть все триггеры привя-

занные к данной базе данных.

Листинг 13. "Создание запроса

```
SELECT* FROM information_schema.triggers where TRIGGER_SCHEMA='dog_db';
```

На рисунке 7 представлен вывод всех 6 триггеров данной базы данных.

| TRIGGER_CATALOG | TRIGGER_SCHEMA | TRIGGER_NAME | EVENT_MANIPULATION | EVENT_OBJECT_CATALOG | EVENT_OBJECT_SCHEMA | EVENT_OBJECT_TABLE |
|---|---|---|---|---|---|---|
| def | dog_db | trigger_breed_update | UPDATE | def | dog_db | breed |
| def | dog_db | trigger_breed_delete | DELETE | def | dog_db | breed |
| def | dog_db | trigger_dog_insert | INSERT | def | dog_db | dog |
| def | dog_db | trigger_dog_update_breed | UPDATE | def | dog_db | dog |
| def | dog_db | trigger_dog_delete | DELETE | def | dog_db | dog |
| def | dog_db | trigger_breed_insert | INSERT | def | dog_db | breed |

Рис. 7. Результат вывода триггеров

В листинге 14 представлена последовательность операций для проверки действия триеггеров. После каждой операции выполняется вывод содержимого таблицы breed_statistics.

Листинг 14. Код для проверки работы триггеров

```


INSERT INTO breed (id_breed, name_breed) VALUES (62, 'Бульбульдог');
SELECT * FROM breed_statistics;

UPDATE breed SET name_breed='Гавгавдог' where id_breed=62;
SELECT * FROM breed_statistics;

INSERT INTO dog (id_dog,name_dog, birth_date,id_gender,number_in_kennel,
    id_kennel,id_breed,id_parents_of_dog) VALUES (22787,'Бобик','2017-05-15'
    ,2, 3000,249,62,20);
SELECT * FROM breed_statistics;

UPDATE dog SET id_breed = 6 WHERE id_dog = 22787;
SELECT * FROM breed_statistics;

DELETE FROM dog where id_dog=22787;
SELECT * FROM breed_statistics;

DELETE FROM breed WHERE id_breed=62;
SELECT * FROM breed_statistics;
```

Результат выполнения каждой операции представлен в таблице 1.

Таблица 1. Перечень тестовых примеров

<table>
<tr><th>№</th><th>user2</th></tr>
<tr><td>1</td><td>В таблицу breed добавляется новая порода Бульбульдог с id_breed=62. При выполнении данной операции в breed_statistics должна добавиться новая порода с количеством собак равным 0. Результат выполнения данной команды с появлением новой породы показан на рисунке 8.

<table>
<tr><th>id_breed</th><th>name_breed</th><th>dog_count</th></tr>
<tr><td>58</td><td>Фокстерьер</td><td>216</td></tr>
<tr><td>59</td><td>Австралийская овчарка</td><td>391</td></tr>
<tr><td>60</td><td>Бордер-терьер</td><td>103</td></tr>
<tr><td>61</td><td>Брюссельский гриффон</td><td>473</td></tr>
<tr><td>62</td><td>Бульбульдог</td><td>0</td></tr>
</table>

Рис. 8. Добавление новой породы</td></tr>
<tr><td>2</td><td>В таблице breed обновляется название только что добавленной 62 породы с Бульбульдог на Гавгавдог. При изменении названия породы должно измениться название соответствующей породы в таблице breed_statistics. Результат выполнения данной операции, представлен на рисунке 9.

<table>
<tr><th>id_breed</th><th>name_breed</th><th>dog_count</th></tr>
<tr><td>58</td><td>Фокстерьер</td><td>216</td></tr>
<tr><td>59</td><td>Австралийская овчарка</td><td>391</td></tr>
<tr><td>60</td><td>Бордер-терьер</td><td>103</td></tr>
<tr><td>61</td><td>Брюссельский гриффон</td><td>473</td></tr>
<tr><td>62</td><td>Гавгавдог</td><td>0</td></tr>
</table>

Рис. 9. Изменение названия породы</td></tr>
<tr><td>3</td><td>В таблицу dog добавляется запись о новой собаке породы Гавгавдог. При выполнении данной операции количество собак породы Гавгавдог должно увеличиться на одну единицу. Результат данной операции представлен на рисунке 10.

<table>
<tr><th>id_breed</th><th>name_breed</th><th>dog_count</th></tr>
<tr><td>58</td><td>Фокстерьер</td><td>216</td></tr>
<tr><td>59</td><td>Австралийская овчарка</td><td>391</td></tr>
<tr><td>60</td><td>Бордер-терьер</td><td>103</td></tr>
<tr><td>61</td><td>Брюссельский гриффон</td><td>473</td></tr>
<tr><td>62</td><td>Гавгавдог</td><td>1</td></tr>
</table>

Рис. 10. Добавление собаки новой породы</td></tr>
<tr><td>4</td><td>Данным действием значение породы только что добавленной собаки изменяется с Гавгавдог на французского бульдога. После данной команды счетчик собак породы Гавгавдог уменьшится на одно значение, а счетчик собак породы французский бульдог наоборот увеличится. Результат данной команды показан на рисунке 11.</td></tr>
</table>

| | |
|---|---|
| | | id_breed | name_breed | dog_count |
|---|---|---|---|
| ▶ | 6 | Французский будльдог | 1 |
| | 62 | Гавгавдог | 0 |

Рис. 11. Изменение породы новой собаки |
| 5 | Запись о данной собаке удаляется из таблицы dog. После удаления количество французских бульдогов уменьшится в таблице breed_statistics уменьшится на один. Результат выполнения данной операции представлен на рисунке 12.

| | id_breed | name_breed | dog_count |
|---|---|---|---|
| ▶ | 6 | Французский будльдог | 0 |
| | 62 | Гавгавдог | 0 |

Рис. 12. Удаление собаки |
| 6 | Последним действием из таблицы breed удаляется запись о породе Гавгавдог. После этого из таблцы breed_statistics так же должна исчезнуть запись о данной породе, что и показано на риснке 13.

| | id_breed | name_breed | dog_count |
|---|---|---|---|
| | 57 | Скотч-терьер | 280 |
| | 58 | Фокстерьер | 216 |
| | 59 | Австралийская овчарка | 391 |
| | 60 | Бордер-терьер | 103 |
| | 61 | Брюссельский гриффон | 473 |

Рис. 13. Удаление породы |

# 4 Права пользователей

## 4.1 Постановка задачи

В данной работе необходимо реализовать ряд следующих задач.

1. Создать пользователя с привилегиями на просмотр таблицы breed_statistics.
2. Создать пользователя с привилегиями на просмотр и изменение таблиц связанных с breed_statistics.
3. Составить сценарий для проверки действия привелегий каждого пользователя.
4. Продемонстрировать работоспособность данного сценария.

## 4.2 Реализация

В данной работе были созданы два новых пользователя user3 и user4. Код создания пользователей представлен в листинге 15. Каждому из пользователей даны права на просмотре данных таблицы breed_statistics. Помимо этого пользователь user4 получил права на просмотр, добавление, изменение и удаление данных из таблиц breed и dog, так как данные таблицы связаны через триггеры с таблицей breed_statistics.

Листинг 15. Создание пользователей user3 и user4

```
CREATE USER IF NOT exists 'user3'@'localhost' IDENTIFIED BY '1234';
GRANT SELECT ON dog_db.breed_statistics TO 'user3'@'localhost';

CREATE USER IF NOT exists 'user4'@'localhost' IDENTIFIED BY '4321';
GRANT SELECT ON dog_db.breed_statistics TO 'user4'@'localhost';
GRANT SELECT, INSERT, UPDATE,DELETE ON dog_db.breed TO 'user4'@'
    localhost';
GRANT SELECT, INSERT, UPDATE,DELETE ON dog_db.dog TO 'user4'@'
    localhost';
```

Для того, чтобы увидеть созданных пользователей и их права доступа к таблицам можно выполнить запрос к information_shema. Данный запрос представлен в листинге 16

Листинг 16. Вывод пользователей и их привелегий

```
SELECT
GRANTEE AS 'User',
TABLE_SCHEMA AS 'Database',
TABLE_NAME AS 'Table',
PRIVILEGE_TYPE AS 'Privilege'
FROM
INFORMATION_SCHEMA.TABLE_PRIVILEGES
WHERE GRANTEE = "'user3'@'localhost'" or GRANTEE = "'user4'@'
    localhost'"
ORDER BY
GRANTEE, TABLE_SCHEMA, TABLE_NAME;
```

В результате данного запроса будет выведена следюущая таблица с пользователями и их правами, представленная на рисунке 14.

| User | Database | Table | Privilege |
|---|---|---|---|
| 'user3'@'localhost' | dog_db | breed_statistics | SELECT |
| 'user4'@'localhost' | dog_db | breed | SELECT |
| 'user4'@'localhost' | dog_db | breed | INSERT |
| 'user4'@'localhost' | dog_db | breed | UPDATE |
| 'user4'@'localhost' | dog_db | breed | DELETE |
| 'user4'@'localhost' | dog_db | breed_statistics | SELECT |
| 'user4'@'localhost' | dog_db | dog | SELECT |
| 'user4'@'localhost' | dog_db | dog | INSERT |
| 'user4'@'localhost' | dog_db | dog | UPDATE |
| 'user4'@'localhost' | dog_db | dog | DELETE |

Рис. 14. Пользователи и их привелегии

Для составления сценария действий пользователей используются команды для проверки работы триггеров из Лабораторной работы 2. В таблице 2 представлены команды и результаты запуска этих команд от лица каждого пользователя. Ниже таблицы представлено пояснение к сценарию, где подробно описываются результаты выполнения представленных на каждом шаге команд.

Таблица 2. Сценарий действий пользователей

| № | Команда | user3 | user4 |
|---|---|---|---|
| 1 | INSERT INTO breed (id_breed, name_breed) VALUES (62, 'Бульбульдог'); | ERROR 1142 (42000): INSERT command denied to user 'user3'@'localhost' for table 'breed' | Query OK, 1 row affected (0.01 sec) |
| 2 | SELECT* FROM breed_statistics; *Команда выполняется после выполнения user2 команды 1. | 60 \| Бордер-терьер \| 103 \|<br>61 \| Брюссельский гриффон \| 473 \|<br>62 \| Бульбульдог \| 0 \|<br>*фрагмент результата запроса. | 60 \| Бордер-терьер \| 103 \|<br>61 \| Брюссельский гриффон \| 473 \|<br>62 \| Бульбульдог \| 0 \|<br>*фрагмент результата запроса. |
| 3 | UPDATE breed SET name_breed='Гавгавдог' where id_breed=62; | ERROR 1142 (42000): UPDATE command denied to user 'user3'@'localhost' for table 'breed' | Query OK, 1 row affected (0.03 sec)<br>Rows matched: 1 Changed: 1 Warnings: 0 |
| 4 | SELECT* FROM breed_statistics; *Команда выполняется после выполнения user2 команды 3. | 60 \| Бордер-терьер \| 103 \|<br>61 \| Брюссельский гриффон \| 473 \|<br>62 \| Гавгавдог \| 0 \|<br>*фрагмент результата запроса. | 60 \| Бордер-терьер \| 103 \|<br>61 \| Брюссельский гриффон \| 473 \|<br>62 \| Гавгавдог \| 0 \|<br>*фрагмент результата запроса. |

| № | Команда | user3 | user4 |
|---|---|---|---|
| 5 | INSERT INTO dog (id_dog, name_dog, birth_date, id_gender, number_in_kennel, id_kennel, id_breed, id_parents_of_dog) VALUES (22787,'Бобик','2017-05-15',2, 3000,249,62,20); | ERROR 1142 (42000): INSERT command denied to user 'user3'@'localhost' for table 'dog' | Query OK, 1 row affected (0.20 sec) |
| 6 | SELECT* FROM breed_statistics; *Команда выполняется после выполнения user2 команды 5. | \| 59 \| Австралийская овчарка \| 391 \|<br>\| 60 \| Бордер-терьер \| 103 \|<br>\| 61 \| Брюссельский гриффон \| 473 \|<br>\| 62 \| Гавгавдог \| 1 \|<br>*фрагмент результата запроса. | \| 59 \| Австралийская овчарка \| 391 \|<br>\| 60 \| Бордер-терьер \| 103 \|<br>\| 61 \| Брюссельский гриффон \| 473 \|<br>\| 62 \| Гавгавдог \| 1 \|<br>*фрагмент результата запроса. |
| 7 | UPDATE dog SET id_breed = 6 WHERE id_dog = 22787; | ERROR 1142 (42000): UPDATE command denied to user 'user3'@'localhost' for table 'dog' | Query OK, 1 row affected (0.03 sec)<br>Rows matched: 1 Changed: 1 Warnings: 0 |
| 8 | SELECT* FROM breed_statistics where id_breed=6 or id_breed=62; *Команда выполняется после выполнения user2 команды 7. | \| 6 \| Французский будльдог \| 1 \|<br>\| 62 \| Гавгавдог \| 0 \| | \| 6 \| Французский будльдог \| 1 \|<br>\| 62 \| Гавгавдог \| 0 \| |
| 9 | DELETE FROM dog where id_dog=22787; | ERROR 1142 (42000): DELETE command denied to user 'user3'@'localhost' for table 'dog' | Query OK, 1 row affected (0.04 sec) |
| 10 | SELECT* FROM breed_statistics where id_breed=6 or id_breed=62; *Команда выполняется после выполнения user2 команды 9. | \| 6 \| Французский будльдог \| 0 \|<br>\| 62 \| Гавгавдог \| 0 \| | \| 6 \| Французский будльдог \| 0 \|<br>\| 62 \| Гавгавдог \| 0 \| |
| 11 | DELETE FROM breed WHERE id_breed=62; | ERROR 1142 (42000): DELETE command denied to user 'user3'@'localhost' for table 'breed' | Query OK, 1 row affected (0.01 sec) |
| 12 | SELECT* FROM breed_statistics; *Команда выполняется после выполнения user2 команды 11. | \| 57 \| Скотч-терьер \| 280 \|<br>\| 58 \| Фокстерьер \| 216 \|<br>\| 59 \| Австралийская овчарка \| 391 \|<br>\| 60 \| Бордер-терьер \| 103 \|<br>\| 61 \| Брюссельский гриффон \| 473 \|<br>*фрагмент результата запроса. | \| 57 \| Скотч-терьер \| 280 \|<br>\| 58 \| Фокстерьер \| 216 \|<br>\| 59 \| Австралийская овчарка \| 391 \|<br>\| 60 \| Бордер-терьер \| 103 \|<br>\| 61 \| Брюссельский гриффон \| 473 \|<br>*фрагмент результата запроса. |

Пояснение к сценарию:

1. **Шаг 1:** Данным действием выполняется добавление новой породы в таблицу breed.

(a) **user3**: Так как данный пользователь не имеет права на добавление записей в таблицу breed, то в консоль выводится соответствующая ошиб-

ка. В результате изменение таблиц breed и breed_statistics не происходит.

(b) **user4**: В результате выполнения данной команды в таблицу breed вставляется новая порода Бульбульдог, так как user4 имеет право на добавление новых записей в таблицу breed.

2. **Шаг 2:** Данная команда демонстрирует результат добавления новой породы вторым пользователем с помощью команды 1. Оба пользователя выводят данные таблицы breed_statistics, так как оба имеют право на просмотр записей данной таблицы. Как видно в таблицу добавлена новая порода со счетчиком равным 0.

3. **Шаг 3:** На данном шаге происходит обновление названия породы Бульбульдог на Гавгавдог.

   (a) **user3**: Так как данный пользователь не имеет права на изменение таблицы breed, то в консоль выводится соответствующая ошибка. В результате изменение таблицы не происходит.

   (b) **user4**: В результате выполнения данной команды в таблице breed изменяется название новой породы, так как user4 имеет право на изменение записей в таблице breed.

4. **Шаг 4:** Данная команда выполняется после того, как user4 изменил название породы командой 3. В результате оба пользователя выводят данные статистики о породах и количестве собак,где видно измененное название породы.

5. **Шаг 5:** На данном шаге в твблицу dog добавляется новая запись о собаке породы Гавгавдог.

   (a) **user3**: Так как данный пользователь не имеет права на добавление записей в таблицу dog, то в консоль выводится соответствующая ошибка. В результате изменение таблиц не происходит.

   (b) **user4**: В результате выполнения данной команды в таблицу dog добавляется собака новой породы, так как user4 имеет право на добавление записей в таблицу dog.

6. **Шаг 6:** Данная команда выполняется после того, как user4 добавил новую собаку командой 5. В результате оба пользователя выводят данные статистики о породах и количестве собак, где счетчик собак для породы Гавгавдог увеличен на 1.

7. **Шаг 7:** Данным действием порода новой собаки меняется с Гавгавдог на французского бульдога.

(a) **user3**: Так как данный пользователь не имеет права на изменение таблицы dog, то в консоль выводится соответствующая ошибка. В результате изменение таблиц не происходит.

(b) **user4**: В результате выполнения данной команды в таблице dog изменяется порода новой собаки ,так как user4 имеет право на изменение записей в таблице dog.

8. **Шаг 8:** Данная команда выполняется после того, как user4 изменил породу собаки командой 7. В результате оба пользователя выводят данные статистики, где счетчик породы Гавгавдог уменьшин на 1, а счетчик французских бульдогов увеличен на 1.

9. **Шаг 9:** На данном шаге происходит удаление собаки из таблицы dog.

(a) **user3**: Так как данный пользователь не имеет права на удаление записей из таблицы dog, то в консоль выводится соответствующая ошибка. В результате изменение таблиц не происходит.

(b) **user4**: В результате выполнения данной команды из таблицы dog удаляется запись о собаке,так как user4 имеет право на удаление записей в таблице dog.

10. **Шаг 10:** Данная команда выполняется после того, как user4 удалил собаку командой 9. В результате оба пользователя выводят данные статистики, где счетчик породы французский бульдог уменьшин на 1.

11. **Шаг 11:** На данном шаге происходит удаление породы Гавгавдог из таблицы breed.

(a) **user3**: Так как данный пользователь не имеет права на удаление записей из таблицы breed, то в консоль выводится соответствующая ошибка. В результате изменение таблиц не происходит.

(b) **user4**: В результате выполнения данной команды из таблицы breed удаляется запись о новой породк,так как user4 имеет право на удаление записей в таблице breed.

12. **Шаг 12:** Данная команда выполняется после того, как user4 удалил породу командой 11. В результате оба пользователя выводят данные статистики о породах и количестве собак, где отсутствует запись об удаленной породе.

На примере данного сценария видно, как привелегии пользователей влияют на выполнение ими соответствующих команд на просмотр, добавление, изменение и удаление записей в рассматриваемых таблицах.

# 5 Функции и процедуры

## 5.1 Постановка задачи

В данной работе необходимо реализовать ряд следующих задач.

1. Написать функцию, которая принимает фаимилию, имя и отчетсво человека и возвращает его фамилию с инициалами.

2. Написать процедуру, которая принимает фамилию,имя и отчество владельца питомника и название питомника. При выполнении данная процедура доложна при необходимости добавлять нового владельца или запись о новом питомнике.

3. Продемонстировать работоспособность написанных функции и процедуры.

## 5.2 Реализация

В данной работе была реализована функция initials для комбинации данных из столбцов с фамилией, именем и отчеством в один столбец с фамилией и инициалами.

Данная функция принимает три переменные varchar разного размера: surname, name и patronymic. Размер переменных определяется из исходной предметной области. При выполнении функции происходит конкатинация фамилии с первыми символами имени и отчества. Отделение первых букв из имени и отчества выполняется с помощью комманды SUBSTRING. Также функция проверяет наличие отчества, так как в других таблицах базы данных данное поле может быть NULL. Реализация данной функция пердставлена в Приложении А.

Проверить выполнение функции можно с помощью простого Select-запроса по выводу данных из таблицы kennel_owner. Данный запрос представлен в листинге 17.

Листинг 17. Проверка функции

```
1 SELECT
2 surname_kennel_owner ,
3 name_kennel_owner,
4 patronymic_kennel_owner ,
5 initials(surname_kennel_owner , name_kennel_owner, patronymic_kennel_owner
    ) AS initials
6 FROM kennel_owner;
```

В результате данного запроса функция применится к каждой строке исходной таблицы и выводится таблица с фамилией, именем, отчеством и их комбинацией. Фрагмент такой таблицы представлен на рисунке 15.

| surname_kennel_owner | name_kennel_owner | patronymic_kennel_owner | initials |
|---|---|---|---|
| Петрова | Алёна | Александровна | Петрова А.А. |
| Сидоров | Николай | NULL | Сидоров Н. |
| Сидорова | Софья | Дмитриевна | Сидорова С.Д. |
| Сидоров | Дмитрий | Сергеевич | Сидоров Д.С. |
| Михайлова | Елена | Дмитриевна | Михайлова Е.Д. |

Рис. 15. Результат вывода работы функции

На данном примере показано, что реализованная функция выполняется корректно.

Также в данной работе было необходимо реализовать процедуру по добавлению записи о питомнике и владельце, если такая запись отсутствует.

Процедура kennel_and_owner принимает на вход данные владельца питомника - фамилию, имя и отчество, а также название питомника. В дальнейшем реализуются три сценария поведения процедуры:

1. Такой владелец питомников существует в таблице kennel_owner. Тогда полученный id владельца передается на следующий этап проверки. Сама таблица kennel_owner не изменяется.

   (a) Существует запись с таким владельцем и указанным питомником. В таком случае процедура не изменяет исходную таблицу.

   (b) Не существует запись с таким владельцем и указанным питомником. В таком случае в таблицу kennel добавляется новая запись, содержащая название питмника и id владельца.

2. Не существует записи о таком владельце в таблице kennel_owner. Тогда в данную таблицу добавляется новый владелец с укзанными ФИО и null значение в поле с номером телефона. После добавления новго владельца в таблицу kennel добавляется новая запись о созданном владельце и питомнике.

Блок-схема данной процедуры представлена на рисунке 16.

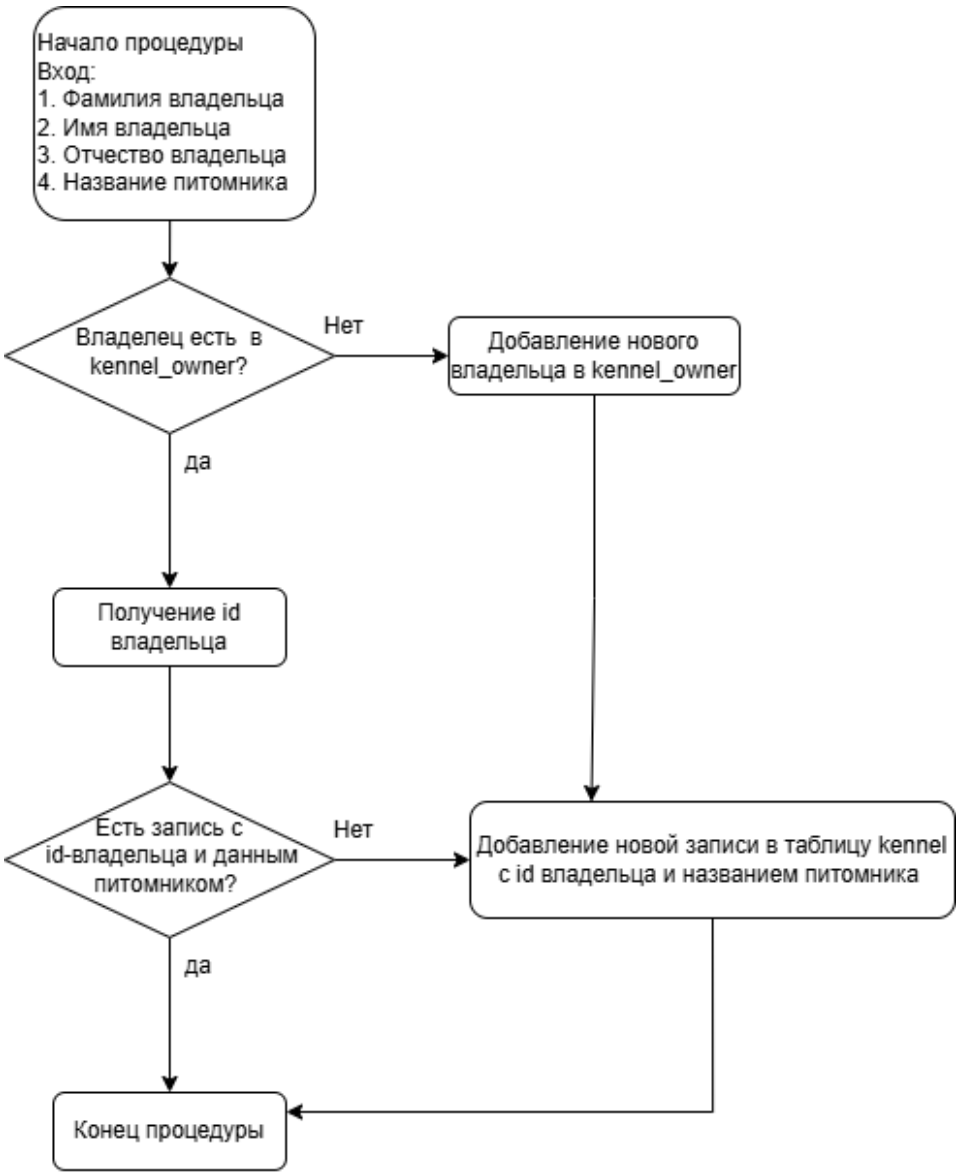


Рис. 16. Блок-схема процедуры

Реализация данной процедуры представлена в Приложении Б.

Проверить результат выполнения данной процедуры можно с помощью разных вызовов процедуры и нескольких запросов, представленных в листингах 18,19 и 20.

В первом вызове в процедуру передаются данные уже существующего владельца и его питомника.

Листинг 18. Запрос 1 для проверки процедуры

```
1 call kennel_and_owner('Питомник1','Петрова','Алёна','Александровна') ;
2  Select* from kennel_owner where surname_kennel_owner ='Петрова' and
     name_kennel_owner =  'Алёна' and patronymic_kennel_owner = 'Александровна'
      ;
3  Select* from kennel where name_kennel = 'Питомник1';
```

В результате выполнения такой процедуры в таблицы kennel и kennel_owner не добавляются новые записи. Так на рисунке 17 показано, что в таблице

kennel_owner так и остается только один владелец с таким ФИО.

| id_kennel_owner | surname_kennel_owner | name_kennel_owner | patronymic_kennel_owner |
|---|---|---|---|
| 1 | Петрова | Алёна | Александровна |
| NULL | NULL | NULL | NULL |

Рис. 17. Результат вывода работы процедуры

На рисунке 18 показано что в таблицу kennel также не добавилась новая запись, а осталась только прежняя запись с данным питомником.

| id_kennel | name_kennel | id_kennel_owner |
|---|---|---|
| 1 | Питомник1 | 1 |
| NULL | NULL | NULL |

Рис. 18. Результат вывода работы процедуры

Второй раз в процедуру передаются данные уже существующего владельца и название питомника, которым он не владеет.

Листинг 19. Запрос 2 для проверки процедуры

```
1
2        call kennel_and_owner('Питомник11','Петрова','Алёна','Александровна') ;
3        Select* from kennel_owner where surname_kennel_owner ='Петрова' and
            name_kennel_owner =  'Алёна' and patronymic_kennel_owner = '
            Александровна' ;
4        Select* from kennel where name_kennel = 'Питомник11' or
            id_kennel_owner=1;
```

В результате выполнения данной процедуры в таблицу kennel_owner не добавится новая запись, а только сохранится прежняя, что и показано на рисунке 19.

| id_kennel_owner | surname_kennel_owner | name_kennel_owner | patronymic_kennel_owner |
|---|---|---|---|
| 1 | Петрова | Алёна | Александровна |
| NULL | NULL | NULL | NULL |

Рис. 19. Результат вывода работы процедуры

В таблицу kennel наоборот добавится новая запись с данным владельцем и названием указанного питомника. Вывод записей измененной таблицы kennel для всех питомников данного владельца и всех питомников с укзаным названием представлен на рисунке 20.

| id_kennel | name_kennel | id_kennel_owner |
|---|---|---|
| 1 | Питомник1 | 1 |
| 2 | Питомник2 | 1 |
| 3 | Питомник3 | 1 |
| 4 | Питомник4 | 1 |
| 5 | Питомник5 | 1 |
| 6 | Питомник6 | 1 |
| 7 | Питомник7 | 1 |
| 8 | Питомник8 | 1 |
| 9 | Питомник9 | 1 |
| 10 | Питомник10 | 1 |
| 11 | Питомник11 | 2 |
| 253 | Питомник11 | 1 |
| NULL | NULL | NULL |

Рис. 20. Результат вывода работы процедуры

В третий раз в процедуру передаются данные несущестующего владельца и название питомника.

Листинг 20. Запрос 1 для проверки процедуры

```
call kennel_and_owner('Питомник250','Петров','Алксей','Александрович') ;
Select* from kennel_owner;
Select* from kennel where name_kennel = 'Питомник250';
```

В реузультате выполнения такого запроса в таблицу с владельцами питомников добавляется новая запись с данным владельцем, что показано на рисунке 21.

| | | | |
|---|---|---|---|
| 29 | Сидорова | Алёна | Дмитриевна |
| 30 | Соколов | Сергей | Андреевич |
| 62 | Петров | Алксей | Александрович |
| NULL | NULL | NULL | NULL |

Рис. 21. Результат вывода работы процедуры

В таблицу с питомниками также добавилась новая запись с созданным владельцем и указанным названием питомника. Вывод такой записи показан на рисунке 22.

| id_kennel | name_kennel | id_kennel_owner |
|---|---|---|
| 254 | Питомник250 | 62 |
| NULL | NULL | NULL |

Рис. 22. Результат вывода работы процедуры

Таким образом на примере данных запросов видно, что процедура корректно обрабатывает разные способы ее вызова.

# 6 Транзакции

## 6.1 Постановка задачи

В данной работе необходимо реализовать ряд следующих задач.

1. Создать двух пользователей для параллельного выполнения транзакций.
2. Выбрать одну аномалию и уровень изоляции транзакций для предотвращения данной аномалии.
3. Написать сценарий действий пользователей для проверки предотвращения аномалии с учетом выбранного уровня изоляции.
4. Продемонстрировать работоспособность данного сценария.

## 6.2 Реализация

В данной работе были созданы два новых пользователя user1 и user2. Код создания пользователей представлен в листинге 21. Первому пользователю были предоставлены права доступа только к таблице breed, а второй пользователь получил права доступа ко всем таблицам базы данных. Данная разница не имеет значения для данной работы, так как оба пользователя будут работать в пределах таблицы breed.

Листинг 21. Создание новых пользователей

```
CREATE USER IF NOT exists 'user1'@'localhost' IDENTIFIED BY '1234';
GRANT SELECT, INSERT, UPDATE ON dog_db.breed TO 'user1'@'localhost';

CREATE USER IF NOT exists 'user2'@'localhost' IDENTIFIED BY '4321';
GRANT SELECT, INSERT, UPDATE ON dog_db.* TO 'user2'@'localhost';
```

В качестве аномалии для изучения было выбрано «грязное чтение» на уровне изоляции READ COMMITED. Данный уровень изоляции позволяет транзакции прочитать данные, измененные в другой транзакции, только после завершения изменившей данные транзакции. Такой уровень изоляции позволяет избежать появление аномалии «грязного чтения», при которой транзакция считывает незафиксированные данные.

На примере созданных пользователей будет рассмотрен сценарий действий для демонстрации предотвращения «грязного чтения» на выбранном уровне изоляции. Данный сценарий и результат выполнения каждого действия представлен в таблице 3. Команды выполняются в параллельно запущенных консолях, в каждой из которых выполнено подключение к базе данных через необходимого пользователя.

Таблица 3. Сценарий действий пользователей

| № | user1 | user2 |
|---|---|---|
| 1 | SET TRANSACTION ISOLATION LEVEL READ COMMITTED;<br><br>START TRANSACTION;<br><br>SELECT * FROM dog_db.breed WHERE id_breed=10;<br><br>Результат запроса:<br>+----------+----------+<br>\| id_breed \| name_breed \|<br>+----------+----------+<br>\|  10 \| Такса  \|<br>+----------+----------+ | - |
| 2 | - | START TRANSACTION;<br><br>UPDATE dog_db.breed SET name_breed='Лабрадудль' WHERE id_breed=10;<br><br>SELECT * FROM dog_db.breed WHERE id_breed=10;<br><br>Результат запроса:<br>+----------+----------+<br>\| id_breed \| name_breed \|<br>+----------+----------+<br>\|  10 \| Лабрадудль \|<br>+----------+----------+ |
| 3 | SELECT * FROM dog_db.breed WHERE id_breed=10;<br><br>Результат запроса:<br>+----------+----------+<br>\| id_breed \| name_breed \|<br>+----------+----------+<br>\|  10 \| Такса  \|<br>+----------+----------+ | - |
| 4 | - | COMMIT; |
| 5 | SELECT * FROM dog_db.breed WHERE id_breed=10;<br><br>Результат запроса:<br>+----------+----------+<br>\| id_breed \| name_breed \|<br>+----------+----------+<br>\|  10 \| Лабрадудль \|<br><br>COMMIT; | - |

Пояснение к сценарию:

1. **Шаг 1:** User1 настраивает уровень изоляции READ COMMITED для транзакции. Затем начинает транзакцию и выполняет запрос на получение информации о породе с id_breed=10. В результате в консоль выводится порода «Такса».

2. **Шаг 2:** User2 начинает свою транзакцию и обновляет название породы с id_breed=10 на название «Лабрадудль». Затем для проверки он выполняет запрос на получение данной породы. В результате выводится обновленное название породы, то есть в пределах данной транзакции user2 будет работать с обновленными данными базы данных.

3. **Шаг 3:** User1 как и в первый раз запрашивает данные о породе с id_breed=10. В результате ему выводится неизменная информация о данной породе, так как транзакция user2 не была завершина, а значит данные не зафиксированы.

4. **Шаг 4:** User2 завершает свою транзакцию и фиксрует изменения.

5. **Шаг 5:** User1 снова запрашивает информацию о той же породе. В результате ему вернется обновленное название породы, так как данные второй транзакции были зафиксированы. Затем пользователь завершает и свою транзакцю.

Таким образом видно, что на уровне изоляции READ COMMITTED не возникает аномалия «грязного чтения»,так как на шаге 3 происходит чтение той же информации, что и на шаге 1, несмотря на то, что на шаге 2 User2 обновляет данные в рамках своей транзакции. При этом User 1 читает обновленную информацию на шаге 5, после того как User2 завершает свою транзакцию на шаге 4.

# Заключение

В ходе данной работы была расширена ранее разработанная база данных с помощью следующих элементов СУБД: представлений, пользователей, триггеров, транзакций, функций и процедур. Работа выполнялась с помощью СУБД MySQK 8.0.

В данной работе было создано представление и с помощью простого и сложного запроса были изучены свойства представлений. Также были созданы триггеры на отображение изменений информации при добавлении, изменении и удалении данных исходных таблиц. Помимо этого в работе были созданы два пользователя с различными правами доступа к таблице breed_statistics. Действия данных пользователей были проверены с помощью составленного сценария. В работе также была написна функция, преобразующая фамилию,имя и отчетсво в фамилию с инициалами. В этой же работы была реализована процедура для работы с ФИО владельца питомника и названием питомника. В последней части работы была изучена анамалия «грязного чтения» на уровне изоляции Read Commited.

Работа выполнялась на ноутбуке HONOR MagicBook X 14 PRO с процессором Intel Core i5. оперативной памятью в 8 Гб и ОС Windows 11.

# Список использованных источников


1. Views in SQL: Syntax and Examples // SQLLer : [сайт]. URL: https://sqller.com/sql/view-sql (дата обращения: 28.11.2025).

2. СУБД: для чего они нужны // Habr : [сайт]. URL: https://habr.com/ru/companies/amvera/articles/754702/ (дата обращения: 28.11.2025).

3. Кузнецов С.Д. Базы данных. Модели и языки. — М.: БИНОМ-ПРЕСС, 2008. — 488 с.

# Приложение А. Исходный код

Листинг 22. Исходный код

```
DELIMITER //
CREATE FUNCTION initials(surname varchar(40) , name varchar(15) ,
   patronymic varchar(20))
RETURNS VARCHAR(45)
DETERMINISTIC
BEGIN
DECLARE results VARCHAR(45) ;
SET results = surname;
IF patronymic IS NOT NULL AND patronymic != '' THEN
SET results = CONCAT(results, ' ', SUBSTRING(name, 1, 1), '.', SUBSTRING(
   patronymic , 1, 1) , '. ');
ELSE
SET results = CONCAT( results , ' ' , SUBSTRING(name, 1, 1), '.' ) ;
END IF;
RETURN results ;
END //
DELIMITER ;
```

# Приложение Б. Исходный код

Листинг 23. Исходный код

```
DELIMITER //
CREATE PROCEDURE kennel_and_owner(kennel_name VARCHAR(20)  ,owner_surname
    VARCHAR(40) ,
        owner_name VARCHAR(15) ,
        owner_patronymic VARCHAR(20))
BEGIN
DECLARE owner_id int ;
DECLARE kennel_id int ;

SELECT id_kennel_owner
INTO owner_id
FROM kennel_owner
WHERE kennel_owner.surname_kennel_owner = owner_surname AND kennel_owner.
    name_kennel_owner =
        owner_name AND kennel_owner.patronymic_kennel_owner =
            owner_patronymic;
IF owner_id IS NULL THEN
        INSERT INTO kennel_owner (surname_kennel_owner, name_kennel_owner,
            patronymic_kennel_owner, phone_kennel_owner)
VALUES ( owner_surname , owner_name , owner_patronymic , NULL);
SET owner_id = LAST_INSERT_ID() ;
END IF;

SELECT id_kennel
INTO kennel_id
FROM  kennel
WHERE name_kennel = kennel_name AND id_kennel_owner=owner_id;

IF kennel_id IS NULL THEN
INSERT INTO kennel (name_kennel, id_kennel_owner)
VALUES (kennel_name , owner_id);
SET kennel_id = LAST_INSERT_ID() ;
END IF;
END //
DELIMITER ;
```